

UNIT VI

6.0 INTRODUCTION

Reading, processing and writing of data are the three essential functions of a computer program. Most programs take some data as input and display the processed data, often known as result. Unlike other high level languages, C does not have any built-in input/output statements as part of its syntax.

A library of functions is supplied to perform these (I/O) operations. The I/O library functions are listed the "header" file <stdio.h>. You do not need to memorize them, just be familiar with them.

The I/O library functions can be classified into two broad categories:

Console I/O functions - functions to receive input from keyboard and write output to VDU.

File I/O functions – functions to perform I/O operations on a floppy disk or a hard disk.

Console I/O Functions

Console I/O refers to the operations that occur at the keyboard and screen of the computer. Because input and output to the console is such a common affair, a subsystem of the ANSI I/O file system was created to deal exclusively with console I/O. Technically, these functions direct their operations to the standard input (**stdin**) and standard output (**stdout**) of the system.

Console I/O functions are further divided into two categories:

1. Formatted console I/O functions —printf/scanf.
2. Unformatted console I/O functions – getchar, putchar etc.

Disadvantages

This works fine as long as the data is small. Real-world problems involve large volumes of data and in such situations Console I/O functions pose two major problems,

1. It becomes time consuming to handle large amount of data.
2. Entire data is lost when either the program is terminated or the computer is turned off.

At these times it becomes necessary to store the data in a manner that can be later retrieved and displayed. This medium is usually a 'file' on the disk. This chapter discusses how file I/O operations can be performed.

6.1 FILES

- ❖ A file is an external collection of related data treated as a unit.
- ❖ The primary purpose of a file is to keep record of data. Record is a group of related fields. Field is a group of characters they convey meaning.
- ❖ Files are stored in auxiliary or secondary storage devices. The two common forms of secondary storage are disk (hard disk, CD and DVD) and tape.
- ❖ Each file ends with an end of file (EOF) at a specified byte number, recorded in file structure.
- ❖ A file must first be opened properly before it can be accessed for reading or writing. When a file is opened an object (buffer) is created and a stream is associated with the object.

6.1.1 BUFFER

- ❖ When the computer reads, the data move from the external device to memory; when it writes, the data move from memory to the external device. This data movement often uses a special work area known as buffer.
- ❖ A buffer is a temporary storage area that holds data while they are being transferred to or from memory.
- ❖ The primary purpose of a buffer is to synchronize the physical devices with a program's need.

6.1.2 FILE NAME

File name is a string of characters that make up a valid filename. Every operating system uses a set of rules for naming its files.

When we want to read or write files, we must use the operating system rules when we name a file. The file name may contain two parts, a primary name and an optional period with extension.

Example: input.txt
 program.c

6.1.3 FILE INFORMATION TABLE

A program that reads or write files needs to know several pieces of information, such as name of the file, the position of the current character in the file, etc.,

C has predefined structure to hold this information. The `stdio.h` header file defines this file structure; its name is `FILE`. When we need a file in our program, we declare it using the `FILE` type.

6.1.4 STREAM

- ❖ A stream is a general name given to a flow of data.
- ❖ All input and output is performed with streams.
- ❖ A "stream" is a sequence of characters organized into lines.
- ❖ Each line consists of zero or more characters and ends with the "newline" character.
- ❖ ANSI C standards specify that the system must support lines that are at least 254 characters in length (including the new line character).
- ❖ A stream can be associated with a physical device, terminal, or with the file stored in memory. The following Figure 6.1 illustrates the data flow between external device(c Program), buffer and file.

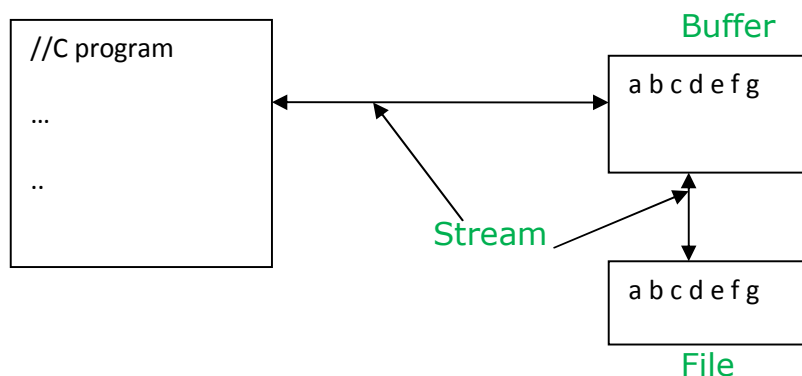


Figure: 6.1 data flow

Text and binary Streams

- ❖ C uses two types of streams: text and binary.
- ❖ A text stream consists of a sequence of characters divided into lines with each line terminated by a new line (`\n`).
- ❖ A binary stream consists of sequence of data values such as integer, real, or complex using their memory representation.

System Created Streams

- ❖ Standard input stream is called "stdin" and is normally connected to the keyboard
- ❖ Standard output stream is called "stdout" and is normally connected to the display screen.
- ❖ Standard error stream is called "stderr" and is also normally connected to the screen.

6.2 STREAM FILE PROCESSING

A file exists as an independent entity with a name known to the O.S. A stream is an entity created by the program. To use a file in our program, we must associate the program's stream name with the file name.

In general, there are four steps to processing a file.

1. Create a stream
2. Open a file
3. Process the file (read or write data)
4. Close file

Creating a Stream

We can create a stream when we declare it. The declaration uses the FILE type as shown below,

```
FILE *fp;
```

The FILE type is a structure that contains the information needed for reading and writing a file, fp is a pointer to the stream.

Opening File

Once stream has been created, we can ready to associate to a file. In the next section we will discuss in detail.

Closing the Stream

When file processing is complete, we close the file. After closing the file the stream is no longer available.

6.3 STANDARD LIBRARY I/O FUNCTIONS

C includes many standard functions to input data from the keyboard and output data to the monitor. For these functions, the stream that connects our programs to the terminal is automatically created by the system and we do not have to do anything more.

The stdio.h header file contains several different input/output functions declarations. These are grouped in to eight different categories, as shown in Figure 6.2.

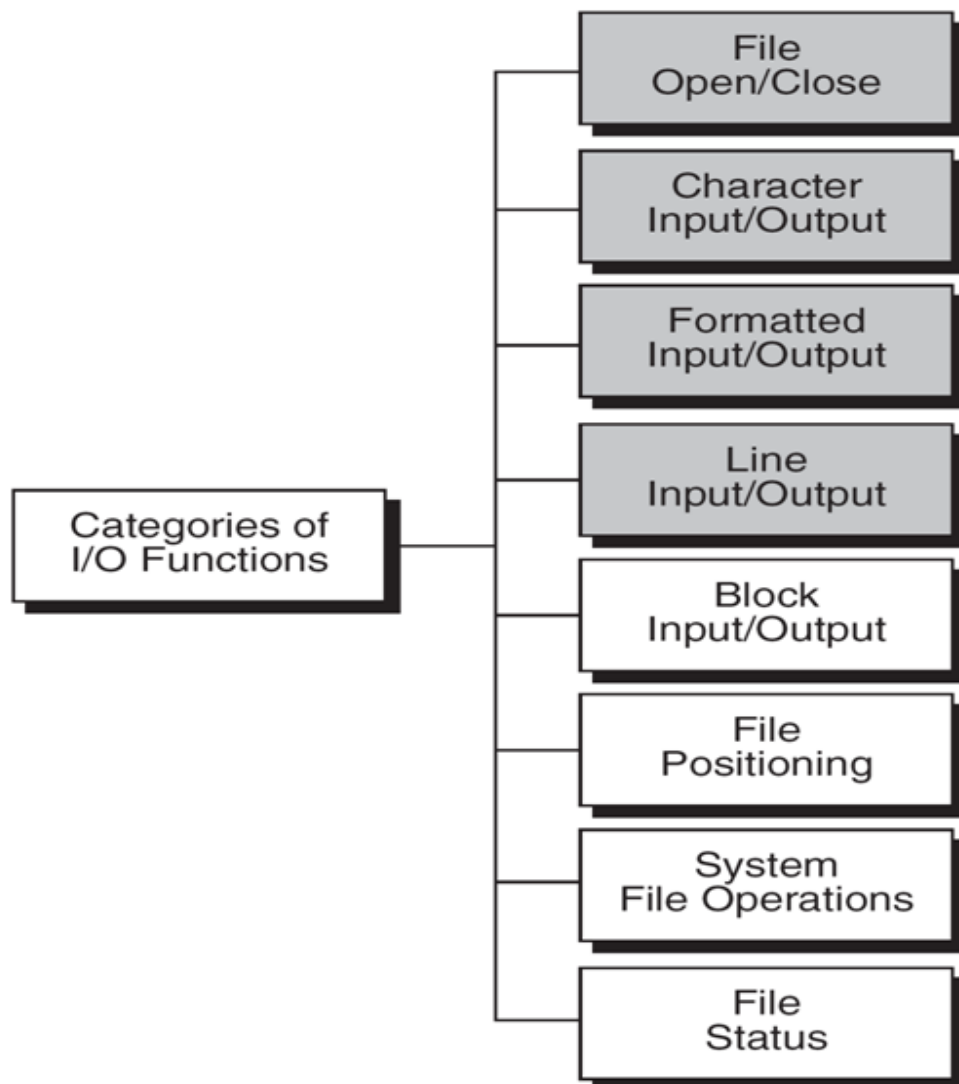


Figure 6.2 Categories of I/O functions

File Open and Close

In this section we discuss the C functions to open and close streams.

File Open (fopen)

The function that prepares a file for processing is fopen. It does two things: first, it makes the connection between the physical file and the file stream in the program. Second, it creates a program file structure to store the information needed to process the file as shown in Figure 6.3

C Program

```
#include<stdio.h>
void main ()
{
....}
```

Buffer

```
abcdefghijklmnpqrst
uvwxyz
```

stream**File Structure**

199	10212	10200	...	...
1	2	3	..	

```
abcdefghijklmnpqrst
uvwxyz
```

stream**File**

Figure 6.3 File open operations

File Structure

Some of the fields of file structure are

- 1) int count: counts bytes left in buffer.
- 2) char *ptr: pointer to the current buffer position.
- 3) char *base: pointer to buffer beginning.

To open a file, we need to specify the physical filename and its mode.

Syntax,

fopen ("filename", "mode");

The file mode is a string that tells C compiler how we intend to use the file: read existing file, write a new file or append to a file.

Successfully opening a file returns a pointer to (i.e., the address of) a file structure. The actual contents of the FILE are hidden from our view. All we need to know is that we can store the address of the file structure and use it to read or write the file.

The statement: `fptr1 = fopen ("mydata", "r");`

Would open the file mydata for input (reading).

The statement: `fptr2 = fopen ("results", "w");`

Would open the file results for output (writing).

Once the files are open, they stay open until you close them or end the program (which will close all files.)

File Mode

When we open a file, we explicitly define its mode. The mode shows how we will use the file: for reading, for writing, or for appending.

"r" (read) mode

The read mode (r) opens an existing file for reading. When a file is opened in this mode, the file marker is positioned at the beginning of the file (first character). The file marker is a logical element in the file structure that keeps track of our current position in the file.

The file must already exist: if it does not, NULL is returned as an error. Files opened for reading are shown in Figure 6.4. If we try to write a file opened in read mode, we get an error message.

Syntax **`fp=fopen ("filename","r");`**

"w" (write) mode

The write mode (w) opens for writing. If the file doesn't exist, it is created. IF it is already exists, it is opened and all its data are erased; the file marker is positioned at the beginning of the file (first character) It is an error to try to read from a file opened in write mode. A file opened for writing is shown in figure 6.4.

Syntax **`fp=fopen ("filename","w");`**

"a" (append) mode

The append mode (a) also opens an existing for writing. Instead of creating a new file, however, the writing starts after the last character; that is new data is added, or appended, at the end of the file.

IF the file doesn't exist, it is created and opened. In this case, the writing will start at the beginning of the file; File opened in append mode are shown in Figure 6.4.

Syntax **fp=fopen ("filename","a");**

"r+" (read and write) mode

In this mode file is opened for both reading and writing the data. If a file does not exist then NULL, is returned.

Syntax: **fp=fopen ("filename","r+");**

"w+" (read and write) mode

In this mode file is opened for both writing and reading the data. If a file already exists its contents erased. If a file does not exist then new file created.

Syntax: **fp=fopen ("filename","w+");**

"a+" (append and read) mode

In this mode file is opened for reading the data as well as data can be added at the end.

Syntax: **fp=fopen ("filename", "a+");**

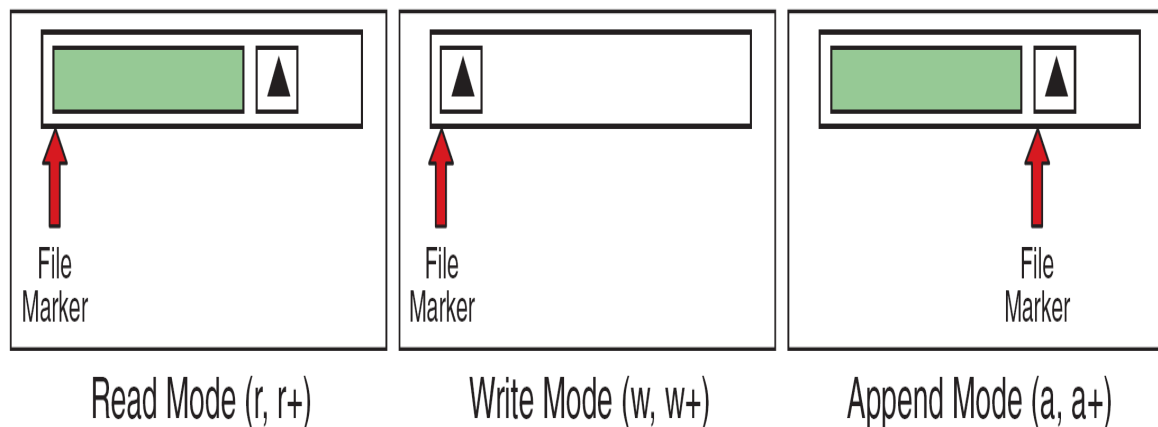


Figure 6.4 File Opening Modes

The above figure shows the file marker position in different modes.

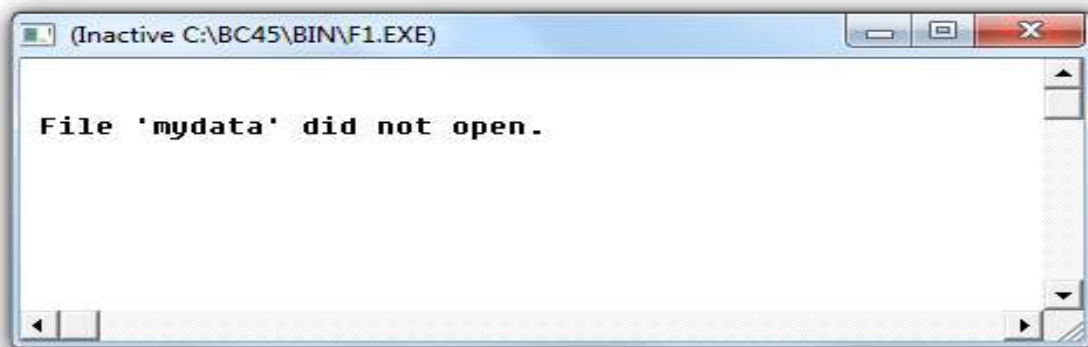
File Close (fclose)

- ❖ Closing a file ensures that all outstanding information associated with the file is flushed out from the buffers and all links to the file are broken.
- ❖ Another instance where we have to close a file is to reopen the same file in a different mode.
- ❖ The I/O library supports the following function to do this:

fclose (file_pointer);

- ❖ Where fp is the file pointer returned by the call to fopen ().
- ❖ fclose () returns 0 on success (or) -1 on error.
- ❖ Once a file is closed, its file pointer can be reused for another file.

```
/*  
  Author: B V Rajesh  
  Date: 20/04/10 23:46  
  Description: Testing for successful open of a file  
*/  
#include <stdio.h>  
int main()  
{  
    FILE *fp ;  
    fp = fopen ( "mydata", "r" ) ;  
    if (fp == NULL)  
    {  
        printf ("\n\n File 'mydata' did not open.\n") ;  
    }  
    fclose(fp);  
    return 0;  
}
```



6.4 FORMATED I/O FUNCTIONS

We have already familiar with two formatting functions scanf and printf. These two functions can be used only with the keyboard and monitor. The C library defines two more general functions, fscanf and fprintf, that can be used with any txt stream.

Formatted Output with printf ()

- ❖ This function provides for formatted output to the screen. The syntax is:

printf ("format string", var1, var2 ...);

- ❖ The "format string" includes a listing of the data types of the variables to be output and, optionally, some text and control character(s).
- ❖ Example:

```
float a=10.5; int b=15;
```

```
printf ("You entered %f and %d \n", a, b);
```

Format Conversion Specifiers

- d -- displays a decimal (base 10) integer
- l -- used with other Specifiers to indicate a "long"
- e -- displays a floating point value in exponential notation
- f -- displays a floating point value
- g -- displays a number in either "e" or "f" format
- c -- displays a single character
- s -- displays a string of characters

Formatted Input with scanf ()

- ❖ This function provides for formatted input from the keyboard. The syntax is:

scanf ("format string" , &var1, &var2, ...) ;

- ❖ The "format string" is a listing of the data types of the variables to be input and the & in front of each variable name tells the system Where to store the value that is input.
- ❖ It provides the address for the variable.

❖ Example:

```
float a; int b;  
scanf ("%f %d", &a, &b);
```

Reading From Files: fscanf ()

❖ The general format of fscanf() is,

fscanf (stream_pointer,"format string", list);

- ❖ The first argument is the stream pointer, is the pointer to the streams that has been declared and associated with a text file. Remaining is same as scanf function arguments.
- ❖ The following example illustrates the use of an input stream.

```
int a, b;  
FILE *fptr1;  
fptr1 = fopen ("mydata", "r");  
fscanf (fptr1, "%d %d", &a, &b);
```

- ❖ The fscanf function would read values from the file "pointed" to by fptr1 and assign those values to a and b.
- ❖ The only difference between scanf and fscanf is that scanf reads data from the stdin (input stream) and fscanf reads input from a user specified stream(stdin or file).
- ❖ The following example illustrates how to read data from keyboard using fscanf,

```
fscanf (stdin,"%d", &a);
```

End of File

The end-of-file indicator informs the program when there are no more data (no more bytes) to be processed. There are a number of ways to test for the

end-of-file condition. Another way is to use the value returned by the fscanf function: The following example illustrates testing of end of file.

```
/*  
  Author: B V Rajesh  
  Date: 21/04/10 00:35  
  Description: Testing for end of file  
*/  
#include<stdio.h>  
int main()  
{  
    int istatus, var ;  
    FILE *fp; // Create stream  
    fp=fopen("input.txt", "a"); // opens a file in append mode  
    istatus = fscanf (fp, "%d", &var) ;  
    if ( istatus == EOF ) // testing for End-Of-File  
    {  
        printf ("\n\n\n End-of-file encountered.\n") ;  
    }  
    fclose(fp); // closing File  
    return 0;  
}
```



Writing To Files: fprintf ()

- ❖ Can handle a group of mixed data simultaneously.
- ❖ The first argument of these functions is a file pointer which specifies the file to be used.
- ❖ The general form of fprintf is

fprintf (stream_pointer, "format string", list);

- ❖ Where stream_pointer is a file pointer associated with a file that has been opened for writing.
- ❖ The format string contains output specifications for the items in the list.
- ❖ The list may include variables, constants and strings.
- ❖ The following example illustrates the use of an Output stream.

```
int a = 5, b = 20;
FILE *fptr2;
fptr2 = fopen ("results", "w");
fprintf (fptr2, "%d %d\n", a, b) ;
```

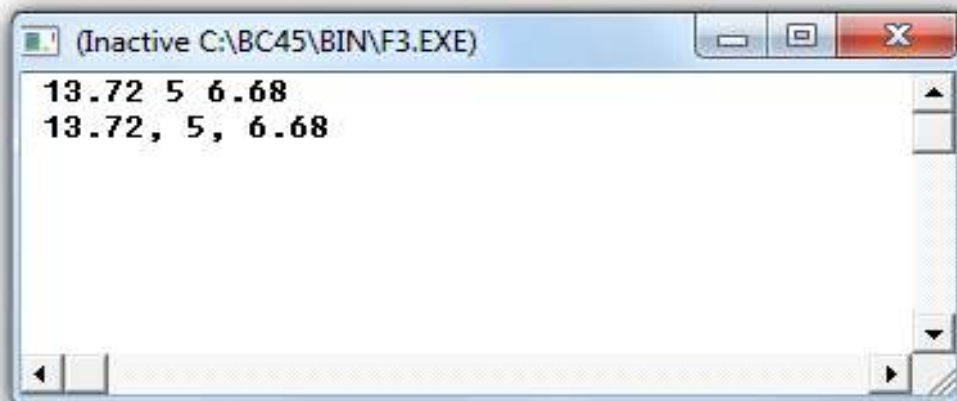
- ❖ The fprintf functions would write the values stored in a and b to the file "pointed" to by fptr2.
- ❖ fprintf function works like printf except that it specifies the file in which the data will be displayed. The file can be standard output (stdout) or standard error (stderr) also.
- ❖ Example,

fprintf (stdout,"%d",45); displays 45 on Monitor.

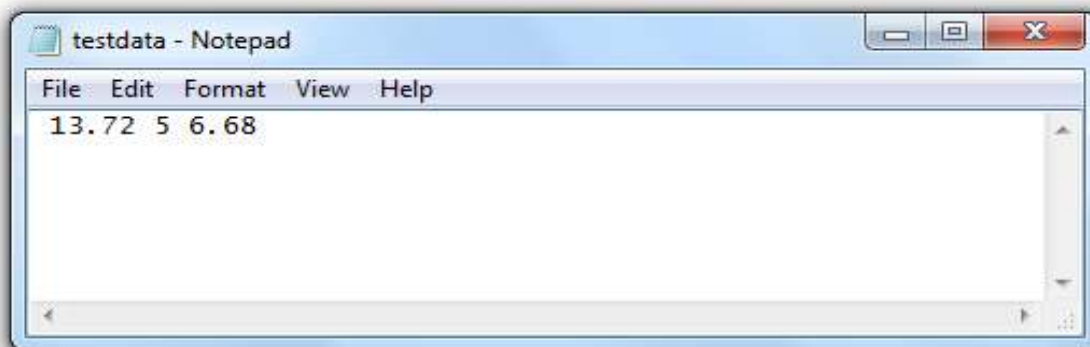
```
/*
  Author: B V Rajesh
  Date: 21/04/10 00:35
  Description: Reading and Writing Files
*/
#include <stdio.h>
int main ( )
{
    FILE *outfile, *infile ;
    int b = 5, f ;
    float a = 13.72, c = 6.68, e, g ;
    outfile = fopen ("testdata.txt", "w") ;
    fprintf (outfile, "%6.2f%2d%5.2f", a, b, c) ;
    fclose (outfile) ;
    infile = fopen ("testdata.txt", "r") ;

    fscanf (infile, "%f %d %f", &e, &f, &g) ;
    printf ("%6.2f%2d%5.2f\n", a, b, c) ;
    printf ("%6.2f,%2d,%5.2f\n", e, f, g) ;
    fclose(infile);
    return 0;
}

// OUTPUT
```

// Contents of File testdata.txt



6.5 CHARACTER I/O FUNCTIONS

Character input functions read one character at a time from a text stream. Character output functions write one character at the time to a text stream.

These functions can be divided into two categories,

1. Terminal Character I/O
2. Terminal and File Character I/O

6.5.1 TERMINAL CHARACTER I/O

C declares a set of character input/output functions that can only be used with the standard streams: standard input (stdin), standard output (stdout).

Read a Character: getchar ()

This function is to read exactly one character from the keyboard; it reads the next character from the standard input stream.

Syntax for using getchar () function is as follows,

```
ch =getchar ();
```

Its return value is integer. Up on successful reading returns the ASCII value of character. If there is an error returns EOF.

Write a Character: putchar ()

This function provides for printing exactly one character to the Monitor.

Syntax for using putchar () function is as follows,

```
ch =getchar (); /* input a character from keyboard*/  
putchar (ch);   /* display character on the Monitor */
```

Its return value is integer. Up on successful writing returns the ASCII value of character. If there is an error returns EOF.

6.5.2 TERMINAL AND FILE CHARACTER I/O

The terminal character input/output functions are designed for convenience; we don't need to specify the stream. Here, we can use a more general set of functions that can be used with both the standard streams and a file.

These functions require an argument that specifies the stream associated with a terminal device or a file.

- ❖ When used with a terminal device, the streams are declared and opened by the system, the standard input stream (stdin) for the keyboard and standard output stream (stdout) for the monitor.
- ❖ When used with a file, we need to explicitly declare the stream, it is our responsibility to open the stream and associate with the file.

Read a Character: getc () and fgetc ()

The getc functions read the next character from the file stream, which can be a user-defined stream or stdin, and converts it into an integer. This function has one argument which is the file pointer declared as FILE.

If the read detects an end of file, the function returns EOF, EOF is also returned if any error occurs. The functionality of getc/fgetc same.

Syntax for using getc/fgetc,

```
getc (fp);  
fgetc (fp);
```

Example,

```
char ch;  
ch = getc (stdin);    /* input from keyboard */  
ch = fgetc (fileptr); /* input from a file */
```

Write a Character: putc () and fputc ()

The putc functions write a character to the file stream specified which can be a user-defined stream, stdout, or stderr. The functionality of putc/ fputc same.

For fputc, the function takes two arguments. The first parameter is the character to be written and the second parameter is the file, The second parameter is the file pointer declared as FILE.

If the character is successfully written, the function returns it. If any error occurs, it returns EOF.

Syntax,

```
putc (char, *fp);  
fputc (char, *fp);
```

Example,

```
char ch;

ch = getc (stdin);    /* input from keyboard */
putc (ch, stdout);    /* output to the screen */
putc (ch, outfileptr); /*output to a file */

/*
  Author: B V Rajesh
  Date: 23/04/10 20:38
  Description: Copy one file contents in to another file
*/

#include<stdio.h>
#include<stdlib.h>
int main()
{
    char ch=1;
    FILE *fptr1,*fptr2;

    /* open file "input.txt" in read mode */
    fptr1=fopen("input.txt","r");
```

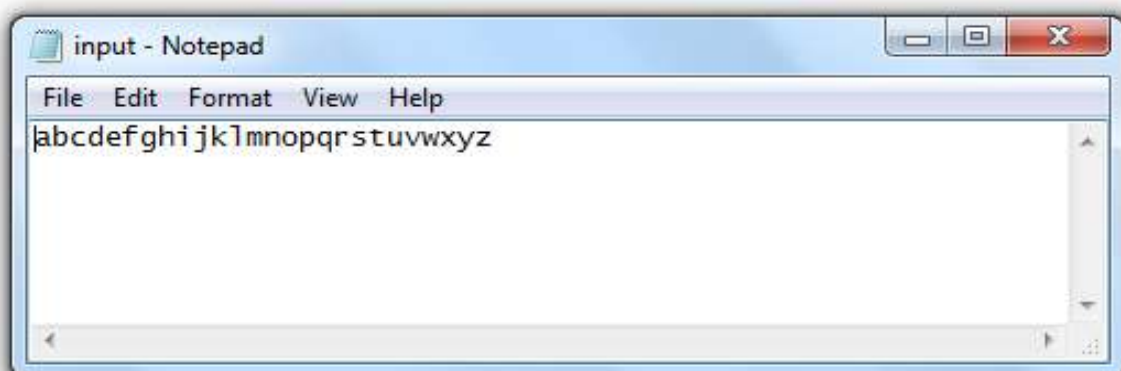
```
if (fptr1 == NULL)
{
    printf ("File 'input.txt' did not open.\n") ;
    exit(1);
}

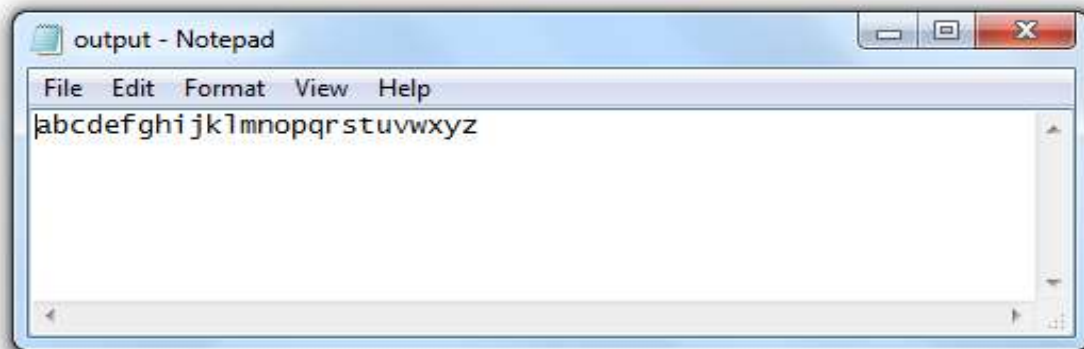
/* open file "output.txt" in write mode */
fptr2=fopen("output.txt","w");

/* Copy file contents */

while(ch!=EOF)
{
    ch=fgetc(fptr1); /* read character from file */
    fputc(ch,fptr2); /* write character to file */
}

fcloseall();
return 0;
}
```





6.6 LINE I/O FUNCTIONS

Reading Strings: gets () and fgets ()

The gets and fgets function take a line (terminated by a new line) from the input stream and make a null-terminated string out of it. These are sometimes called line-to-string input function.

gets ()

- ❖ gets function reads a string from terminal, reading is terminated by new line.
- ❖ The newline (\n) indicates the end of the string, it is replaced by null (\0) character in the memory by the function gets () as shown in Figure 6.5.
- ❖ The source of data for the gets function is standard input.
- ❖ gets function takes only one parameter, the name of the string, its syntax is as follows,

gets (string_name);

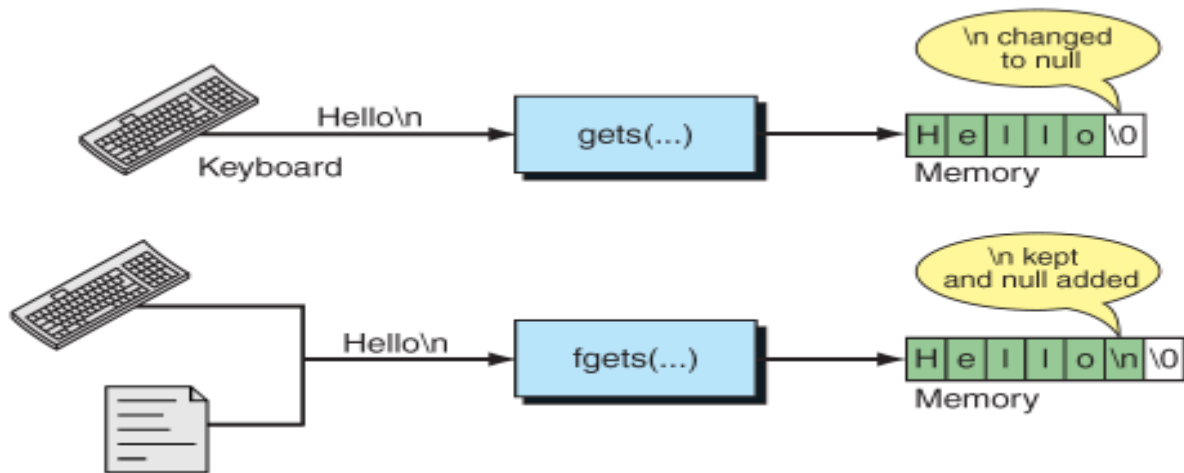


Figure 6.5 Working of gets/fgets functions

fgets ()

- ❖ fgets function reads a string from file or keyboard.
- ❖ It reads characters from the specified file until a new line character has been read or until n-1 characters has been read, whichever occurs first. The function automatically places null character at the end as shown in Figure 6.5.
- ❖ The source of data for fgets can be a file or standard input.
- ❖ The general format,

fgets (string, length, fp);

- ❖ First argument is name of the string in which the read data from the file is to be placed. Second argument is the length of the string to be read and last argument is the file pointer.
- ❖ fgets also reads data from terminal. Below example illustrates this,

fgets (str, 10, stdin);

```
/*
  Author: B V Rajesh
  Date: 23/04/10 23:30
  Description: Illustrates working of gets/fgets
*/

#include<stdio.h>
#include<stdlib.h>

int main()
{
    char str1[20],str2[20];
    FILE *fp;

    /* open file "getdata.txt" in read mode */

    fp=fopen("getdata.txt","r");

    if (fp == NULL)
    {
        printf ("File 'getdata' did not open.\n") ;
        exit(1);
    }
    printf("\n Enter a string: ");
    gets(str1); /* read a string from keyboard */

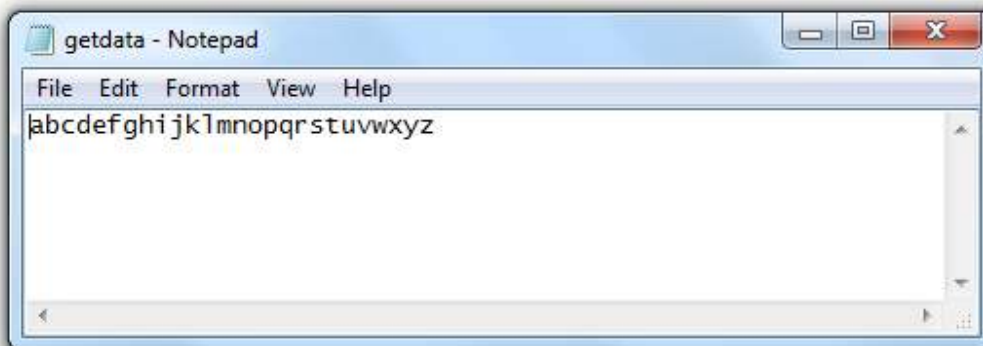
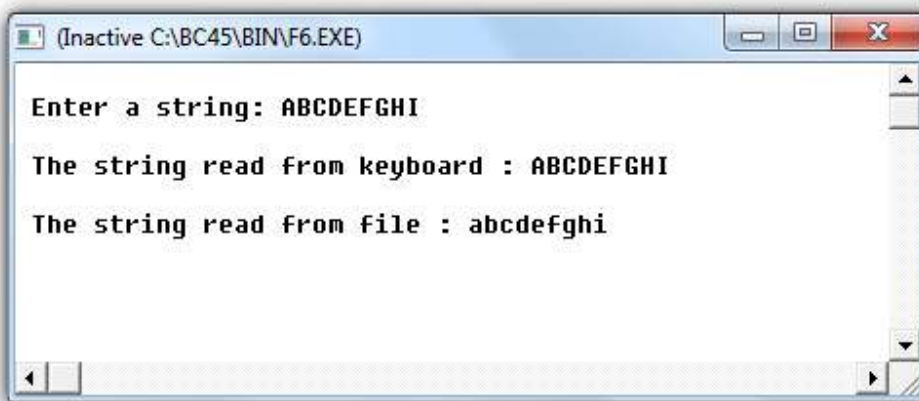
    fgets(str2,10,fp); /* read a string from file */
}
```



```
fgets(str2,10,fp); /* read a string from file */

printf("\n The string read from keyboard : %s",str1);
printf("\n\n The string read from file : %s",str2);

fclose(fp);
return 0;
}
```



Writing Strings: puts () and fputs ()

puts/fputs functions take a null-terminated string from memory and write it to a file or the monitor. These are sometimes called string-to-line output functions.

puts ()

- ❖ puts function writes a string to terminal, writing is terminated by null character. the null character is replaced with a new line inputs as shown in Figure 6.6.
- ❖ puts function takes only one parameter, the name of the string, its syntax is as follows,

puts (string_name);

fputs ()

- ❖ fputs function writes a string to a file or monitor.
- ❖ Characters stored in the string are written to the file identified by fileptr until the null character is reached. The null character is not written to the file as shown in Figure 6.6.
- ❖ The general format forms are

fputs (string, fp);

- ❖ First argument is address of the string in which the read data is to be placed. Second argument is the file pointer.

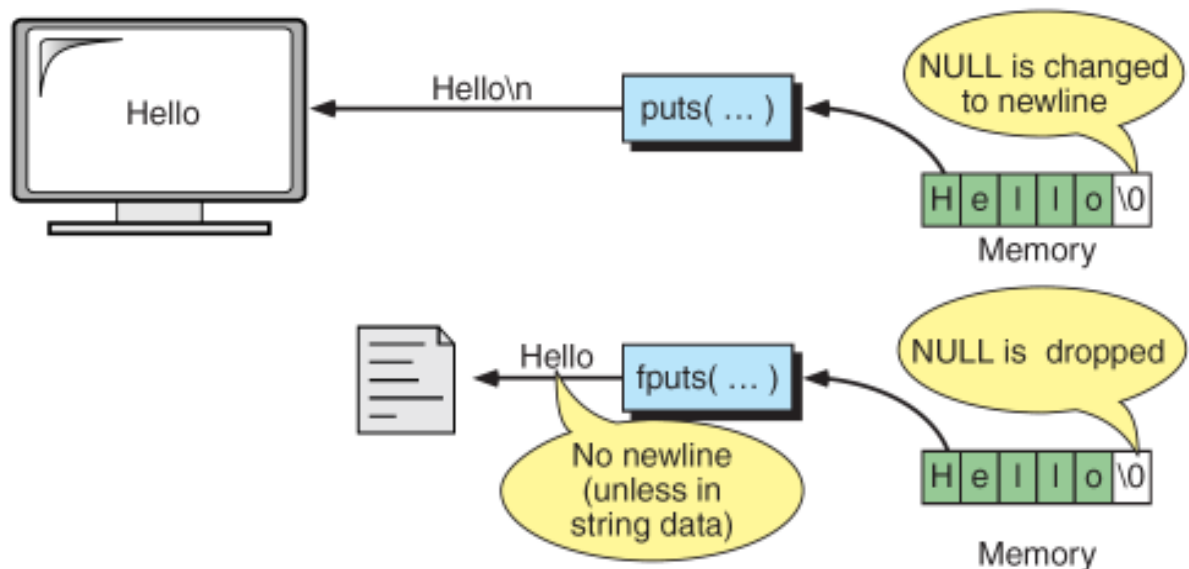
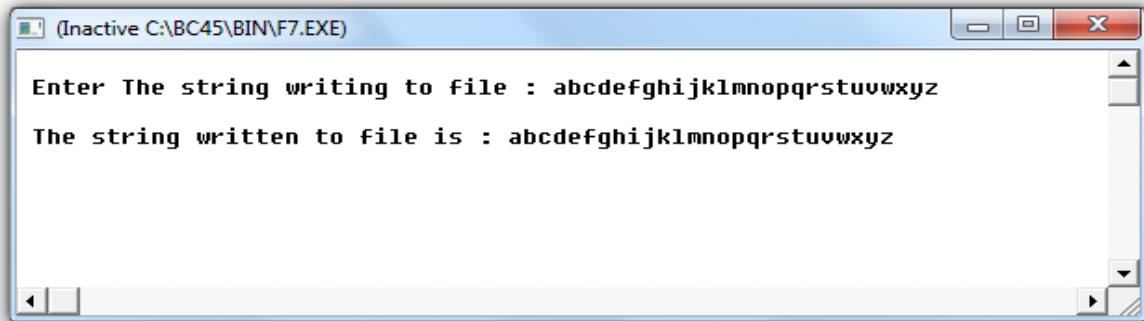


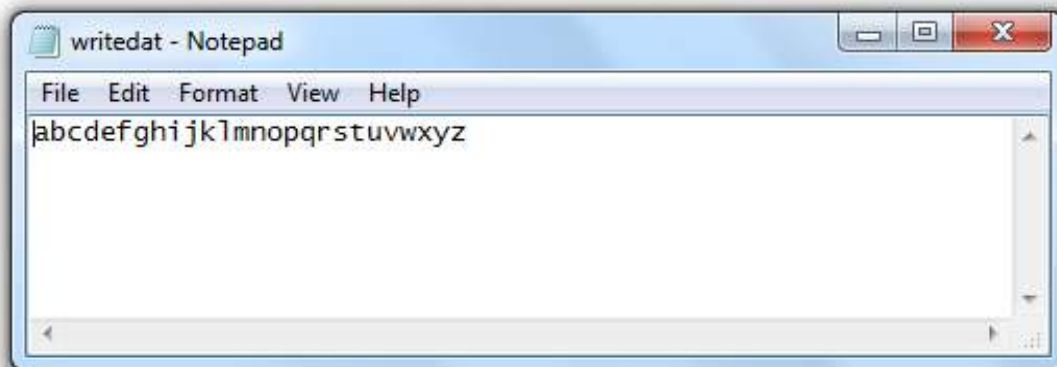
Figure 6.6 Working of puts/fputs Functions

```
/*  
  
    Author: B V Rajesh  
    Date: 24/04/10 00:02  
    Description: Illustrates puts/fputs  
*/  
#include<stdio.h>  
#include<stdlib.h>  
  
int main()  
{  
    char str[50];  
    FILE *fp;  
  
    /* open file "writedata.txt" in update mode */  
  
    fp=fopen("writedata.txt","w+");  
  
    if (fp == NULL)  
    {  
        printf ("File 'writedata' did not open.\n") ;  
        exit(1);  
    }  
  
    printf("\n Enter The string writing to file : ");  
    gets(str);  
  
    fputs(str,fp); /* Write a string of characters */  
  
    fputs(str,fp); /* Write a string of characters */  
  
    printf("\n The string written to file is : ");  
    puts(str);  
  
    fclose(fp);  
    return 0;  
}
```

// output



// Contents of file writedat.txt



6.6.1 STRING/DATA CONVERSION FUNCTIONS

A common set of applications format data by either converting a sequence of characters into corresponding data types or vice versa. C already has set of data conversion functions created for scanf and printf. The C standard also includes two sets of memory formatting functions: sscanf and sprintf.

String to Data Conversion: sscanf ()

The string scan function is called sscanf. sscanf is a one-to-many function. It splits one string into many variables.

This function scans a string through the data were coming from a file. Just like fscanf, it requires a format string to provide the formatting parameters for the data.

Instead of reading these data from the keyboard, we can also read from a string stored in memory using `sscanf ()`.

Syntax,

`sscanf (string, "control string", variables);`

The first argument is the string, which contains the data to be scanned. The second is the control string consisting of format Specifiers. Variables, it is a list of variables in to which formatted data is copied.

Its return value is integer. If the `sscanf ()` is successful it returns how many variables formatted. The error is specified with EOF.

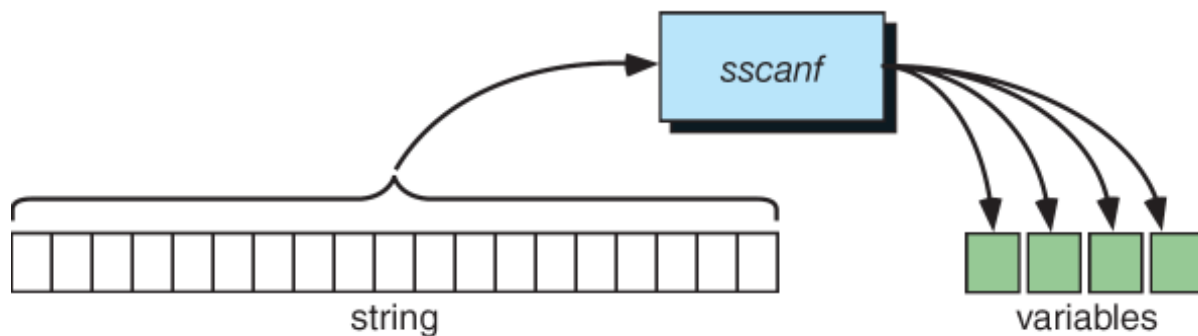


Figure 6.7 Working of sscanf Function

Example,

```
sscanf ("hyd 45 67.89", "%s %d %f", name, &a, &b);
```

Would result in name=hyd, a=45 and b=67.890000

Data to String Conversion sprintf ()

The string print function is `sprintf`, follows the rules of `fprintf`. Rather than sending the data to a file. however, it simply "writes" them to a string. When all data have been formatted to the string, a terminating null character is added to make the result a valid string. If an error is detected, `sprintf` returns any negative value or EOF. If the formatting is successful, it returns number of characters formatted.

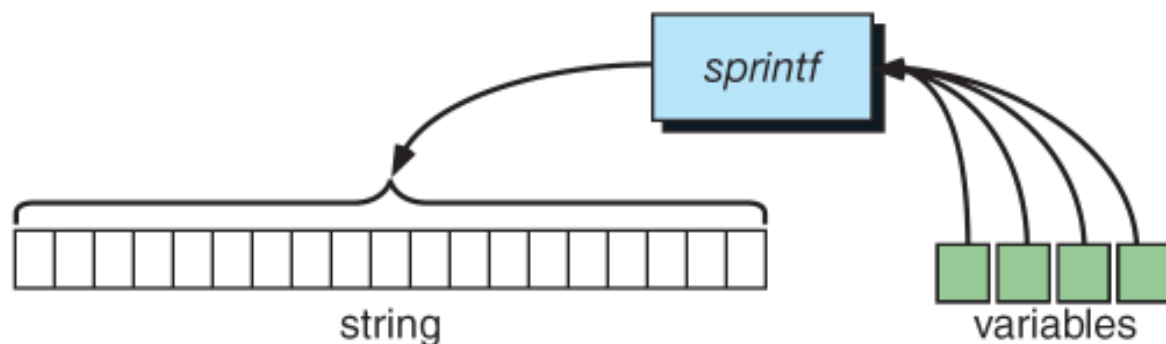


Figure 6.8 Working of sprintf Function

Syntax,

sprintf (string, "control string", variables);

The first argument is the string, which contains the data to be written. The second is the control string consisting of format Specifiers. Variables, it is a list of variables in to which formatted data is copied.

Example,

```
sprintf (str,"%s %d %f", name, a, b);
```

will result in str =name + a + b.

```
/*
  Author: B V Rajesh
  Date: 28/04/10 17:32
  Description: Illustrating sscanf/sprintf functions
*/
#include<stdio.h>
int main()
{
    char str[40]="Einstein.Albert 1000 98.45";
    char str1[40];
    char name[20];
    int id;
    float avg;

    printf("\n The contents of String is:%s",str);
```

```

    sscanf(str, "%s%d%f", name, &id, &avg);

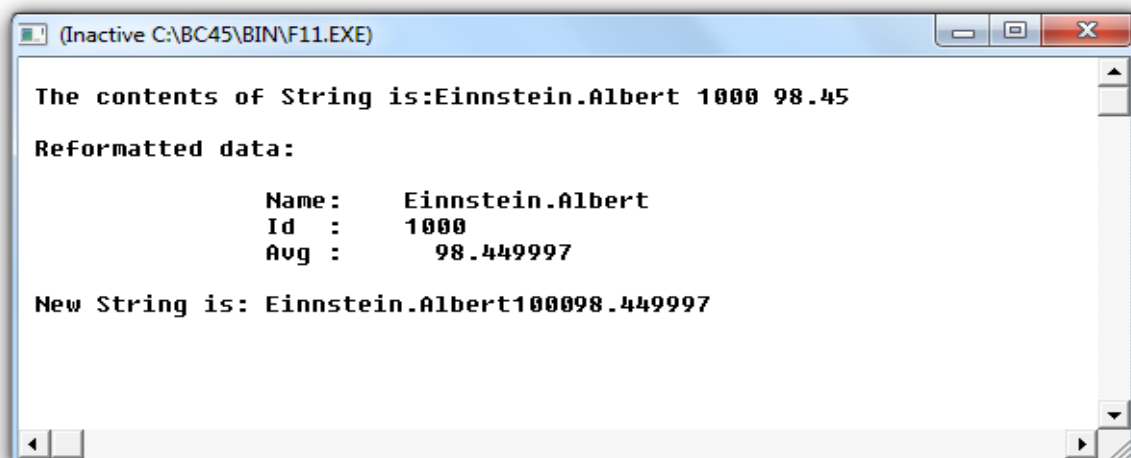
    printf("\n\n Reformatted data: ");
    printf("\n\n\t\tName:      %s", name);
    printf("\n\t\tId       :      %d", id);
    printf("\n\t\tAvg      :      %f", avg);

    sprintf(str1, "%s%d%f", name, id, avg);

    printf("\n\n New String is: %s", str1);

    return 0;
}

```



6.7 TEXT FILES AND BINARY FILES

Text File

A text file is the one where data is stored as a stream of characters that can be processed sequentially. In the text format, data are organized into lines terminated by newline character. The text files are in human readable form and they can be created and read using any text editor. Since text files process only characters, they can read or write only one character at a time. A new line may be converted to carriage return and line feed character. Figure 6.9 shows the data transfer in text file.

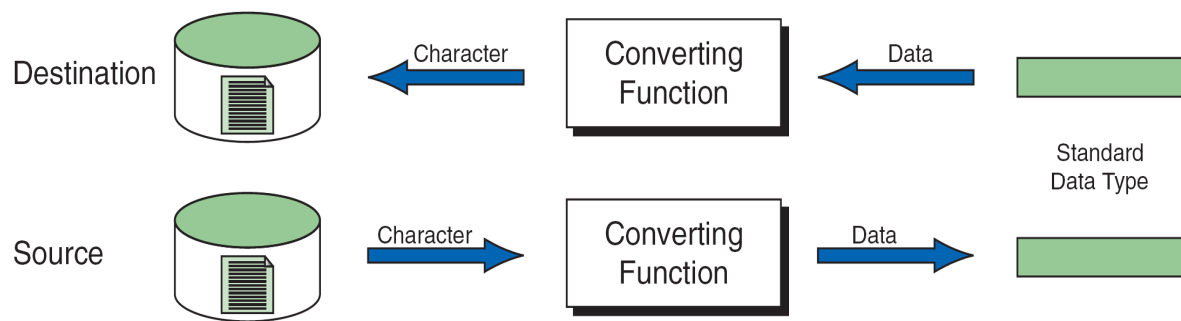


Figure 6.9 Reading and Writing Text Files

Because of these translations, the number of characters written / read may not be the same as the number of characters written / read on the external device. Therefore, there may not be a one – to – one relationship between the characters written / read into the file and those stored on the external devices.

For example, the data 2345 requires 4 bytes with each character occupying exactly one byte. A text file cannot contain integers, floating – point numbers etc. converted to their character – equivalent formats.

Binary File

A binary file is the one where data is stored on the disk in the same way as it is represented in the computer memory. The binary files are not in human readable form and they can be created and read only by specific programs written for them. The binary data stored in the file cannot be read using any of the text editors.

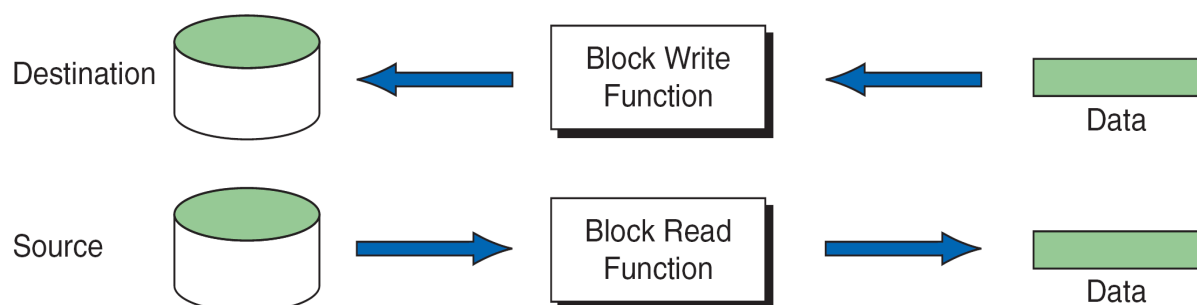


Figure 6.10 Block Input and Output

For example, the data 2345 takes 2 bytes of memory and is stored as 0 x 0929 i.e., 0 x 09 is stored as one byte and 0 x 29 is stored as other byte. The number of characters written / read is same as the number of characters written / read on the external device. Therefore, there is one to – one relationship between the characters written / read into the file and those stored on the external devices.

Differences between Text File and Binary File

The differences between text file and binary file are as shown in Table 6.1. The following Figure 6.11 illustrates the storage difference between text file and binary file.

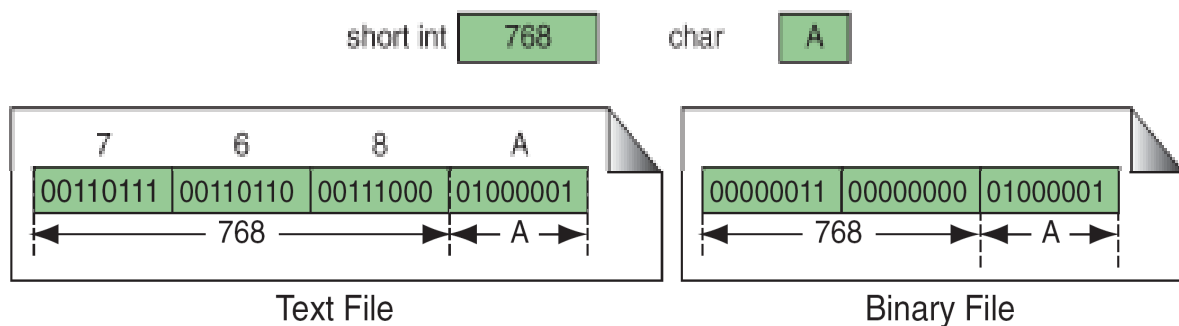


Figure 6.11 Binary and Text Files

<i>Text File</i>	<i>Binary File</i>
1. Human readable format.	1. Not in human readable format.
2. Data is stored as lines of characters with each line terminated by / n which may be translated into carriage return + line feed.	2. Data is stored on the disk in the same way as it is represented in the computer memory.
3. Translation of new line character to carriage return + line feed.	3. No translation of any type.
4. Data can be read using any of the text editors.	4. Data can be read only by specific programs written for them.

5. <i>The number of characters written / read may not be the same as the number of characters written/read on the external device such as disk.</i>	5. The number of characters written / read is same as the number of characters written / read on the external device such as disk.
6. <i>There may not be a one – to- one relationship between the characters written / read into the file and those stored on the external devices.</i>	6. There is one – to – one relationship between the characters written / read into the file and those stored on the external devices.
7. The data 2345 requires 4 bytes with each character requiring one byte. So, it is stored as 0x 32, 0x 33, and 0 x 34, 0 x 35 (ASCII values) thus requiring 4 bytes.	7. The data 2345 requires 2 bytes and stored as 0x0929. The data 2345 is requiring 2 bytes.

Table 6.1 Differences between Text File and Binary File

Opening Binary Files

The basic operation is unchanged for binary files-only the mode changes. Different modes of operation of binary file are illustrated in Table 6.2.

Mode	Meaning
wb	This mode opens a binary file in write mode. Example: fp=fopen ("data.dat","wb");
rb	This mode opens a binary file in read mode. Example: fp=fopen ("data.dat","rb");
ab	This mode opens a binary file in append mode. Example: fp=fopen ("data.dat","ab");

w+b	This mode opens/creates a binary file in write and read mode. Example: <code>fp=fopen ("data.dat","w+b");</code>
r+b	This mode opens a pre-existing binary file in read and write mode. Example: <code>fp=fopen ("data.dat","r+b");</code>
a+b	This mode opens/creates a binary file in append mode. Example: <code>fp=fopen ("data.dat","a+b");</code>

Table 6.2 Binary Modes of Opened File

Just like text files, binary files must be closed when they are not needed anymore using `fclose ()`.

6.8 BLOCK I/O FUNCTIONS

C language uses the block input and output functions to read and write data to binary files. As we know that data are stored in memory in the form of 0's and 1's. When we read and write the binary files, the data are transferred just as they are found in memory and hence there are no format conversions. The block read function is `file read(fread)`. The block write function is `file write(fwrite)`.

File Read: `fread ()`

This function reads a specified number of bytes from a binary file and places them into memory at the specified location. This function declaration is as follows,

`int fread (void *pInArea, int elementsize, int count, FILE *sp);`

The first parameter, `pInArea`, is a pointer to the input area in memory. The data read from the file should be stored in memory. For this purpose, it is required to allocate the sufficient memory and address of the first byte is stored in `pInArea`. We say that `pInArea` now points to buffer.

The next two elements, `elementSize` and `count`, are multiplied to determine how much data are to be transferred. The size is normally specified using the `sizeof` operator and the count is normally one when reading structures. The last argument is the pointer to the file we want to read from.

This function returns the number of items read. If no items have been read or when error has occurred or EOF encountered, the function returns 0.

File read expects a pointer to the input area, which is usually a structure. This is because binary files are most often used to store structures.

Figure 6.12 is an example of a file read that reads data into an array of integers. When `fread` is called, it transfers the next three integers from the file to the array, `inArea`.

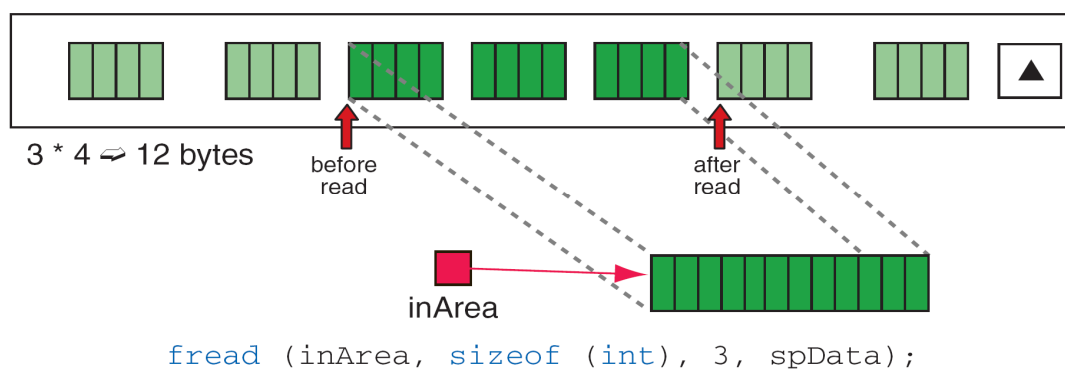


Figure 6.12 File Read Operation

File Write: fwrite ()

This function writes specified number of items to a binary file. This function declaration is as follows,

int fwrite (void *pInArea, int elementsize, int count, FILE *sp)

The parameters for file write correspond exactly to the parameters for the file read function. Figure 6.13 shows the write operation that parallels the read in Figure 6.12.

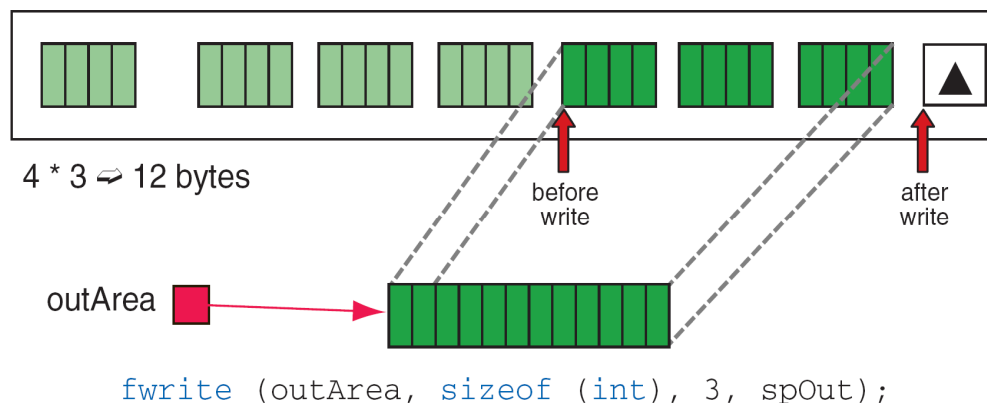


Figure 6.13 File Write Operation

6.9 FILE POSITIONING FUNCTIONS

File positioning functions have two uses. First, for randomly processing data in disk files, we need to position the file to read the desired data. Second, we can use the positioning functions to change a file's state. Thus, we can change the state of the file pointer using one of the positioning functions.

There three file position functions,

- ❖ tell location
- ❖ rewind file
- ❖ file seek

Current Location: ftell ()

- ❖ The ftell function reports the current position of the file marker in the file, relative to the beginning of the file.
- ❖ ftell () takes a file pointer and returns a number of type long integer, that corresponds to the current position.
- ❖ Here the return type is long integer because many files have more than 32,767 bytes. The operation ftell is graphically shown in Figure 6.14.
- ❖ The syntax of ftell function as follows,

n= ftell(fp);

- ❖ n would give the relative offset of the current position. This means that n bytes have already been read (or written).
- ❖ If ftell encounters an error, it returns -1.
- ❖ Let us consider the following statement,

`n=ftell (fp);`

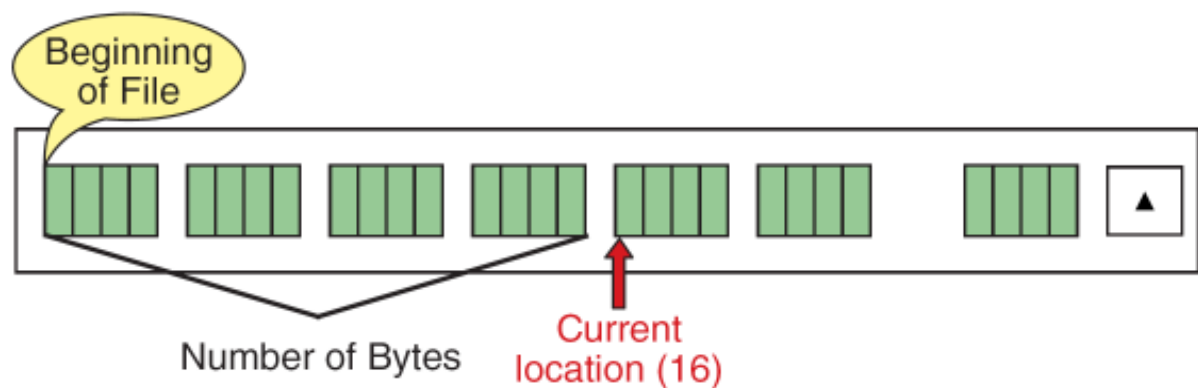


Figure 6.14 Working of ftell Function

- ❖ The above Figure 6.14 illustrates the working of ftell, it returns 16, and the current offset value.
- ❖ The primary purpose of ftell is to provide a data address (offset) that can be used in a file seek. It is especially helpful when we are dealing with text file for which we cannot calculate the position of data.

Rewind File: rewind ()

- ❖ The rewind function simply sets the file position indicator to the beginning of the file as shown in Figure 6.15.
- ❖ This function helps us in reading a file more than once, without having to close and open the file.
- ❖ Its common use is to change a work from a write state to a read state. However, that to read and write a file with only one open, we must open it in update mode w+.
- ❖ Syntax to use rewind function is as follows,

rewind (fp);

❖ Example:

```
rewind (fp);
```

```
n=ftell (fp);
```

Would assign 0 to n because the file position has been set to the start of the file by rewind.

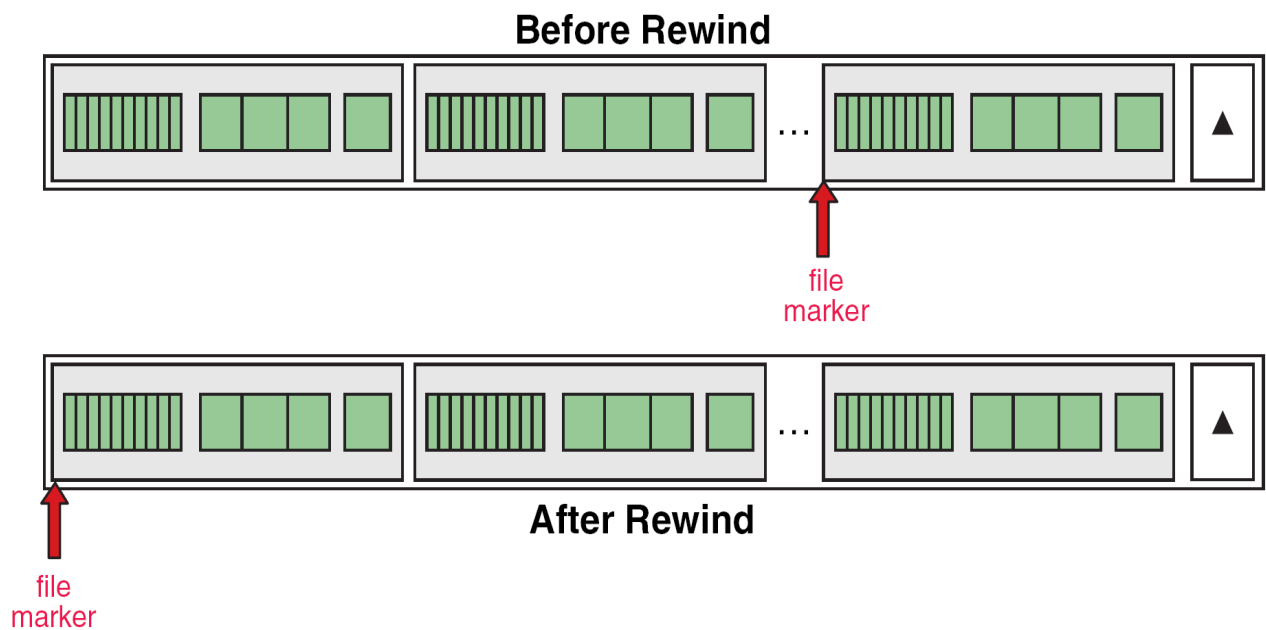


Figure 6.15 Working of `ftell` Function

Set Position: `fseek ()`

- ❖ `fseek ()` function is used to move the file position to a desired location within the file.
- ❖ It takes the following form:

`fseek (file_ptr, offset, position);`

- ❖ `file_ptr` is a pointer to the file concerned, `offset` is a number or variable of type long, and `position` is an integer number.

- ❖ The offset specifies the number of positions to be moved from the location specified by position.
- ❖ The position can take one of the following three values:

Value	Meaning
0	beginning of file.
1	Current position.
2	End of file.

- ❖ The offset may be positive, meaning move forwards, or negative, meaning move backwards.
- ❖ The following Table 6.3 illustrate operations of the fseek() function:

Statement	Meaning
<i>fseek(fp,0L,0);</i>	go to the beginning.
<i>fseek(fp,0L,1);</i>	stay at the current position.
<i>fseek(fp,0L,2);</i>	go to the end of the file, past the last character of the file.
<i>fseek(fp,m,0)</i>	move to (m+1) th byte in the file.
<i>fseek(fp,m,1);</i>	go forward by m bytes.
<i>fseek(fp,-m,1);</i>	go backward by m bytes from the current position.
<i>fseek(fp,-m,2);</i>	go backward by m bytes from the end. (positions the file to the character from the end.)

Table 6.3 Operations of the fseek function

- ❖ When the operation is successful, fseek returns a zero.
- ❖ If we attempt to move the file pointer beyond the file boundaries, an error occurs and fseek returns -1.

```

/*
  Author: B V Rajesh
  Date: 28/04/10 18:31
  Description: Illustrating file positioning functions
*/
#include<stdio.h>
int main()
{
    FILE *fp;
    char arr[30];
    int i=0;
    long int n;

    // open input.txt in read mode
    fp=fopen("input.txt","r");

    n=ftell(fp); // tells current position of file marker
    printf("\nThe current position of file marker=%ld ",n);

    fseek(fp,-5,2); //go backward by 5 bytes from the end

    n=ftell(fp); //tells current position of file marker
    printf("\nThe current position of file marker=%ld ",n);

    printf("\n\n Characters read from file are: ");

    for(i=0;i<5;i++)
    {
        arr[i]=fgetc(fp);
        fputc(arr[i],fp);
        printf("%c",arr[i]);
    }
}

```

```
    }

    rewind(fp); // reset offset to begining

    n=ftell(fp); //tells current position of file marker
    printf("\nThe current position of file marker=%ld ",n);

    //goforward by 5 bytes from current position
    fseek(fp,5,1);

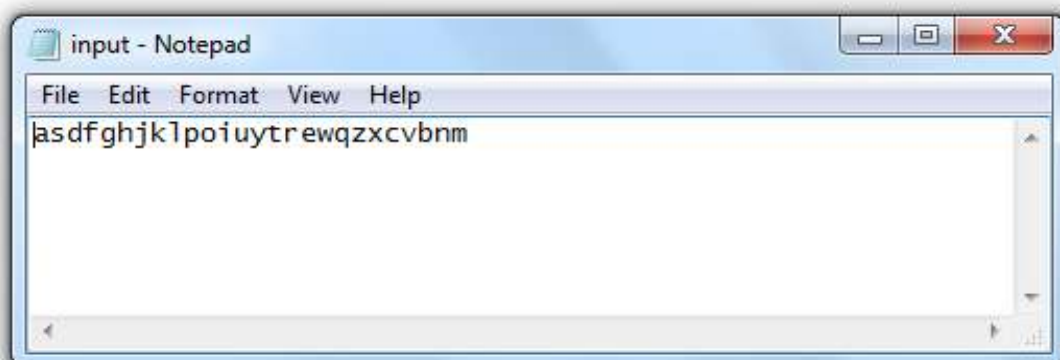
    printf("\n\n Characters read from file are: ");
    for(i=0;i<5;i++)
    {
        arr[i]=fgetc(fp);
        fputc(arr[i],fp);

        printf("%c",arr[i]);
    }

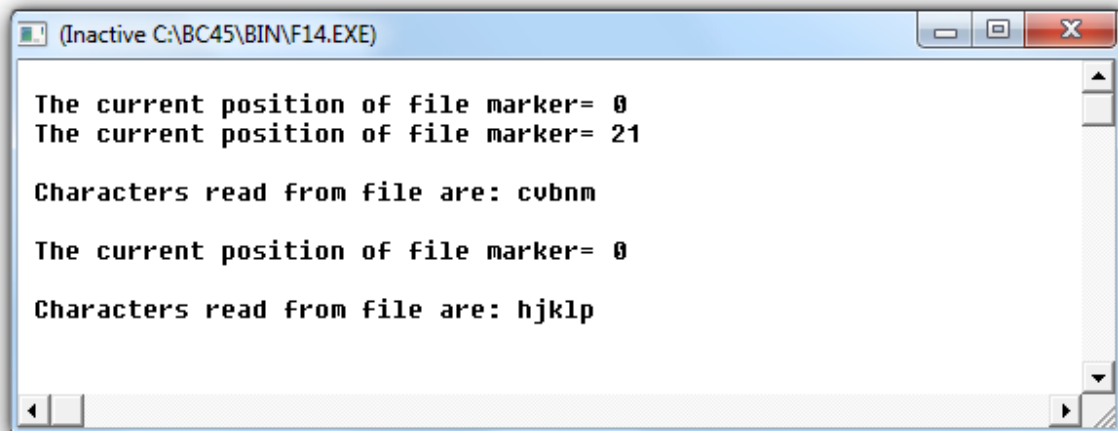
    fclose(fp);

    return 0;
}
```

// contents of input.txt



// output

A screenshot of a Windows command window titled "(Inactive C:\BC45\BIN\F14.EXE)". The window contains the following text:

```
The current position of file marker= 0
The current position of file marker= 21

Characters read from file are: cvbnm

The current position of file marker= 0

Characters read from file are: hjklp
```

6.10 FILE STATUS FUNCTIONS

C provides functions for error handling during I/O operations. Typical error situations include:

- ❖ Trying to read beyond the end-of-file mark.
- ❖ Device overflow.
- ❖ Trying to use a file that has not been opened.
- ❖ Trying to perform an operation on a file, when the file is opened for another type of operation.
- ❖ Opening a file with an invalid filename.
- ❖ Attempting to write to a write-protected file.

We have two status-inquiry library functions, `feof ()` and `ferror ()` that can help us detect I/O errors in the files.

Test End of Files: `feof ()`

- ❖ The `feof` function can be used to test for an end of file condition.
- ❖ It takes a `FILE` pointer as its only argument. If all data have been read, the function returns a nonzero (positive value) and returns zero (false) otherwise.

- ❖ It takes of the following form,

n=feof (fp);

- ❖ If fp is a pointer to file that has just been opened for reading, then the statement

if (feof (fp))

printf ("End of data.\n");

- ❖ Would display the message "End of data" on reaching the end of file condition.

Test Error: ferror ()

- ❖ The ferror function reports the status of the file indicated. Error can be created for many reasons, such as trying to read a file in the write state.
- ❖ It takes of the following form.

n=ferror (fp);

- ❖ It also takes a FILE pointer as its argument and returns a non zero integer if an error has been detected up to that point, during processing. Otherwise, it returns zero.
- ❖ The statement

if(ferror(fp)!=0)

printf ("An error has occurred.\n");

- ❖ Would print the error message, if the reading is not successful.

However, that testing for an error not reset the error condition. Once a file enters the error state. It can only return to a read or write state by calling clear error function: **clearerr**.

It takes of the form

clearerr (fp); // clears the error information

6.11 COMMAND LINE ARGUMENTS

Command line arguments are parameters supplied to a program, when the program is invoked.

C language provides a way to connect to the arguments on the command line needed for the execution of the program. During execution, arguments can be passed to the main () function through command-line arguments. The first parameter treated as name of the file.

In order to access the command-line arguments, the main function has a prototype as shown below,

int main (int argc, char* argv [])

The first parameter argc stands for the argument count, which is of integer data type. Its value is the number of arguments in the command line that was used to execute the program.

The second parameter argv stands for the argument vector. It is an array of pointers to the character data type. The program name is the first parameter on the command line, which is argv [0].

// C program illustrates Command Line Arguments

```
#include<stdio.h>

int main (int argc, char *argv [])
{
    int j;
    printf ("The name of the program is %s", argv[0]);
    printf ("The total number of arguments are: %d", argc);
    for (j=1; j<=argc; j++)
        printf ("\n argument %d is %s', j, argv[j]);
    return 0;
}
```

OUTPUT:

C:\tc\bin\>test one two three

The name of the program is test

The total number of arguments are:4

argument 1 is one

argument 2 is two

argument 3 is three

Exercise Programs

1. Write a program to read the following data, to find the value of each item and display the contents of the file.

Item	Code	Price	Quantity
Pen	101	Rs. 20	5
Pencil	103	Rs. 3	100

2. Write a C program to read a text file and to count
 - (a) number of characters,
 - (b) number of words and
 - (c) number of sentences and write in an output file.
3. Write a program to open a pre-existing file and add information at the end of file. Display the contents of the file before and after appending.
4. Write a C program to read last 'n' characters of the file using appropriate file function.
5. Write a C program to read a text file and convert the file contents in capital (upper-case) and write the contents in a output file.
6. Write a C program to replace every 5th character of the data file, using fseek () command.
7. Write a program to read the contents of three files and write the file with largest number of characters into an output file.
8. Write a C program to read information about the student record containing student's name, student's age and student's total marks. Write the marks of each student in an output file.

ASSIGNMENT VI

1.
 - a) Distinguish between text mode and binary mode operation of a file.
 - b) Write short notes on Block I/O Functions
2. What are the file I/O functions in C. Give a brief note about the task performed by each function?
3. Explain the following operations
 - a) fseek()
 - b) ftell()
 - c) rewind()
 - d) ferror()
 - e) feof()
4. Write the syntax for opening a file and closing a file. Explain different modes of operation of a file.